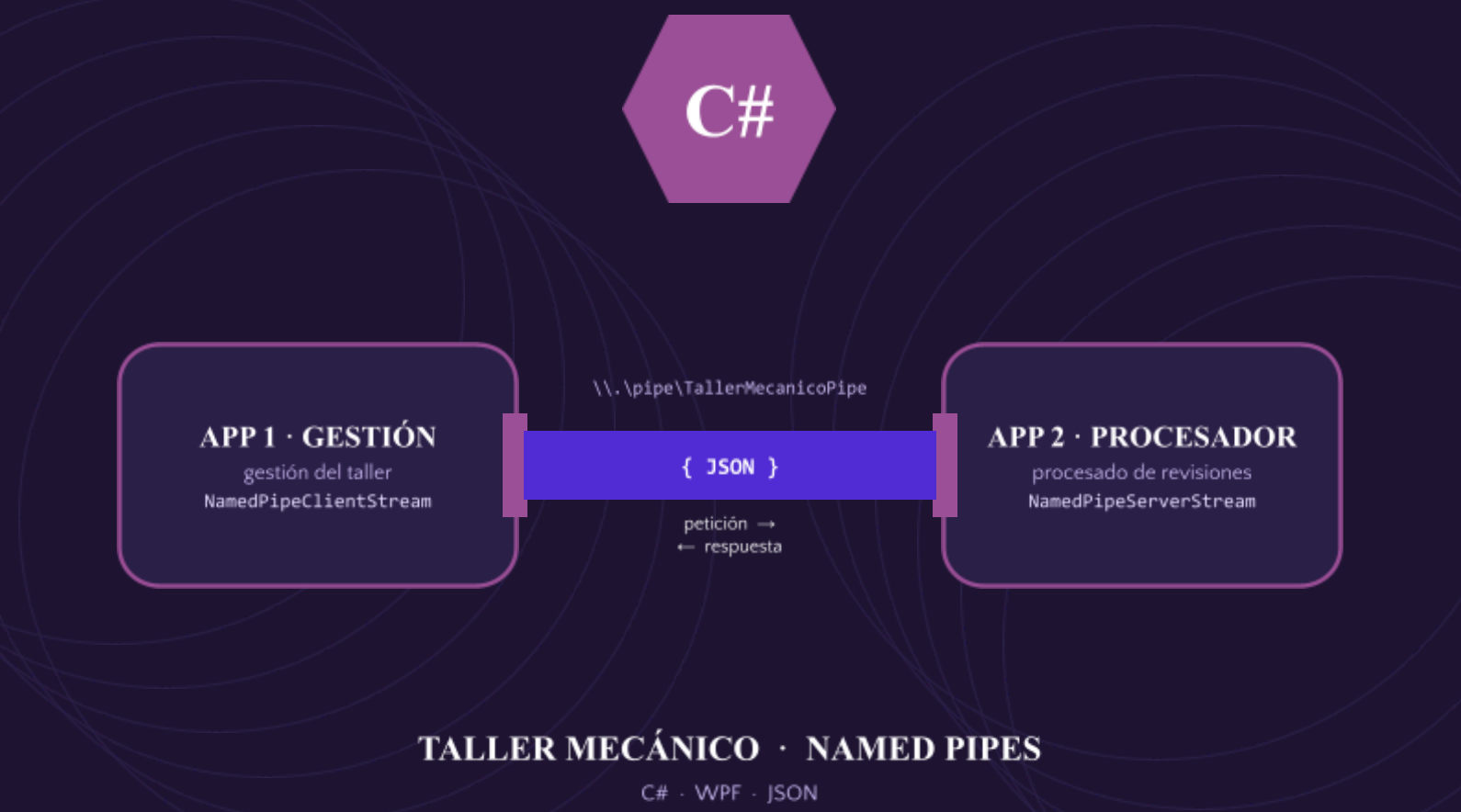


DESARROLLO DE INTERFACES · PROGRAMACIÓN DE SERVICIOS Y PROCESOS

NAMED PIPES EN C#

COMUNICACIÓN ENTRE LAS APLICACIONES DEL TALLER

DIEGO GONZÁLEZ · DAVID CORDÓN
2º DAM · Curso 2026-2027

Desafío PSP y DIN · Taller mecánico
Fecha de entrega: __ / __ / 2026

ÍNDICE

1. Introducción **3**

2. ¿Qué son las Named Pipes? **4**

2.1. Servidor y cliente 4

2.2. Comparación con otros mecanismos de comunicación 4

3. Named Pipes en C# **5**

3.1. Clases y métodos principales 5

3.2. Ejemplo de código 5

4. Comunicación en el taller mecánico **6**

4.1. Funcionamiento paso a paso 6

5. Formato de los datos (JSON) **7**

5.1. Petición (App 1 → App 2) 7

5.2. Respuesta (App 2 → App 1) 7

5.3. Registro de peticiones 7

5.4. Descripción de los campos 7

6. Dificultades y soluciones **8**

7. Conclusiones **9**

8. Bibliografía **10**

1. INTRODUCCIÓN

En este trabajo vamos a investigar qué son los Named Pipes en C# y cómo funcionan. Se trata de un sistema que permite que dos programas se comuniquen entre sí y se intercambien información.

Primero explicaremos cómo funcionan y qué ventajas tienen. Después veremos cómo podemos utilizarlos en nuestro proyecto, cómo se comunicarán los dos programas y qué información se enviarán entre ellos.

También pondremos algún ejemplo de código en C# y de cómo podría ser el JSON que utilizamos para enviar los datos. Así podremos entender mejor cómo funcionan los Named Pipes y cómo aplicarlos a nuestro proyecto.

2. ¿QUÉ SON LAS NAMED PIPES?

Las named pipes son mecanismos IPC que permite que dos o mas aplicaciones se comuniquen entre sí, funcionan tanto entre procesos que corren en la misma máquina como entre equipos diferentes a través de una red y pueden configurarse para enviar y recibir datos en ambas direcciones o en una sola dirección.

2.1. Servidor y cliente

SERVIDOR

El servidor es el creador y administrador de la tubería, y se encarga de gestionar el canal de comunicación, algunas de sus funciones son:

- Crea la tubería, Define el ID de la pipe en el sistema y configura los permisos, la dirección de los datos (unidireccional o bidireccional) y el número máximo de instancias.
- Escucha activamente, se pone en modo espera usando el método WaitForConnection() bloqueando el hilo de ejecución hasta que un cliente se conecte
- Seguridad, decide que usuarios o procesos tienen permisos a conectarse a la tubería
- Cerrar el canal, cuando termina la comunicación, el servidor desconecta al cliente y libera el recurso

CLIENTE

El cliente es el iniciador de la conversación. No tiene control sobre la tubería sus funciones son:

- Conectar, con el nombre de la tubería, intenta localizarla en la máquina local y solicita la conexión con el método Connect().
- Especifica tiempo de espera, al intentar conectarse al servidor, el cliente puede definir cuánto tiempo esperará si el servidor está ocupado antes de lanzar un error
- Recibe y envía datos, una vez aceptado, empieza a leer o escribir datos según los permisos otorgados por el servidor

FLUJO DE INTERACCIÓN

1. El servidor se crea
2. El cliente trata de conectarse
3. El servidor lo acepta
4. Intercambio de datos
5. El cliente cierra la conexión
6. El servidor limpia el canal

2.2. Comparación con otros mecanismos de comunicación

Mecanismo	Cómo funciona	Ventajas	Inconvenientes
Named Pipes	Canal con nombre (\\.\pipe\Nombre) entre un servidor y uno o varios clientes. En C# se usa con System.IO.Pipes.	Rápido en el mismo equipo, bidireccional (InOut), no necesita puertos ni red y permite trabajar con async/await.	Pensado para el mismo equipo o una red Windows. Hay que respetar el orden de lectura y escritura para no bloquearse.
Anonymous Pipes	Tubería sin nombre entre un proceso padre y el proceso hijo que él mismo lanza (AnonymousPipeServerStream).	Muy sencilla y ligera.	Solo funciona entre padre e hijo y en un único sentido. No sirve para dos apps independientes como las nuestras.
Sockets TCP	Conexión mediante una dirección IP y un puerto (TcpListener y TcpClient).	Funciona entre equipos distintos y con otros sistemas operativos, incluso por Internet.	Más código, hay que elegir un puerto y el cortafuegos puede bloquearlo.
Archivos compartidos	Una app escribe un fichero en una carpeta y la otra lo lee.	Fácil de entender y de comprobar, porque el fichero se puede abrir.	No avisa cuando hay datos nuevos, da problemas si las dos apps lo usan a la vez y es más lento.
Memoria compartida	Las dos apps leen y escriben en una misma zona de memoria (MemoryMappedFile).	Es la opción más rápida cuando se mueven muchos datos.	Hay que controlar a mano quién escribe en cada momento (por ejemplo con un Mutex) y es más compleja.

3. NAMED PIPES EN C#

.

3.1. Clases y métodos principales

Clase / método	Para qué sirve
NamedPipeServerStream	Crea la tubería del servidor y permite esperar conexiones de los clientes.
NamedPipeClientStream	Permite al cliente conectarse a la tubería del servidor.
PipeDirection.InOut	Permite enviar y recibir información por la misma tubería.
PipeTransmissionMode.Byte	Envía los datos como una secuencia de bytes.
PipeOptions.Asynchronous	Permite trabajar de forma asíncrona para evitar bloqueos mientras se espera una operación.
WaitForConnectionAsync()	Hace que el servidor espere a que un cliente se conecte sin bloquear el hilo.
ConnectAsync(5000)	Intenta conectar el cliente al servidor y establece un tiempo máximo de espera de 5 segundos.
StreamReader	Permite leer los mensajes recibidos por la tubería.
StreamWriter	Permite escribir y enviar mensajes por la tubería.
ReadLineAsync()	Lee un mensaje hasta encontrar un salto de línea, sin bloquear el hilo mientras espera.
WriteLineAsync()	Envía un mensaje terminado en un salto de línea de forma asíncrona.
async y await	Permiten ejecutar operaciones asíncronas y esperar a que terminen sin bloquear el hilo que las ejecuta.
Task.Run()	Permite ejecutar el servidor en segundo plano para evitar que la ventana WPF se quede congelada.
Dispatcher.InvokeAsync()	Permite actualizar elementos de la interfaz WPF desde otro hilo.
JsonSerializer.Serialize()	Convierte un objeto de C# en una cadena de texto JSON para enviarlo.
JsonDocument.Parse()	Permite leer e interpretar un mensaje JSON recibido.
PipeOptions	Permite configurar opciones de funcionamiento de la tubería, como la comunicación asíncrona

3.2. Ejemplo de código

C# · SERVIDOR (APP 2)

```
using System;
using System.IO;
using System.IO.Pipes;

namespace NamedPipeServer
{
    class Program
    {
        static void Main(string[] args)
        {
            using (var server = new NamedPipeServerStream("MyPipe"))
            {
                Console.WriteLine("Esperando conexión...");
                server.WaitForConnection();

                using (var reader = new StreamReader(server))
                {
                    string line = reader.ReadLine();
                    Console.WriteLine($"Recibido del cliente: {line}");
                }
            }
        }
    }
}
```

C# · CLIENTE (APP 1)

```
using System;
using System.IO;
using System.IO.Pipes;

namespace NamedPipeClient
{
    class Program
    {
        static void Main(string[] args)
        {
            using (var client = new NamedPipeClientStream(".", "MyPipe", PipeDirection.Out))
            {
                client.Connect();

                using (var writer = new StreamWriter(client))
                {
                    writer.AutoFlush = true;
                    writer.WriteLine("¡Hola, servidor!");
                }
            }
        }
    }
}
```

4. COMUNICACIÓN EN EL TALLER MECÁNICO

La App 1 (gestión) coge las facturas de las reparaciones y se las envía a la App 2 por la Named Pipe. La App 2 tiene un límite de facturación fijado en su código: si un mecánico supera ese límite, le calcula un incentivo proporcional a su sueldo y devuelve el resultado a la App 1, que lo guarda en la base de datos. Encaja bien con el enunciado, porque la App 2 hace un cálculo interno sin tener base de datos.

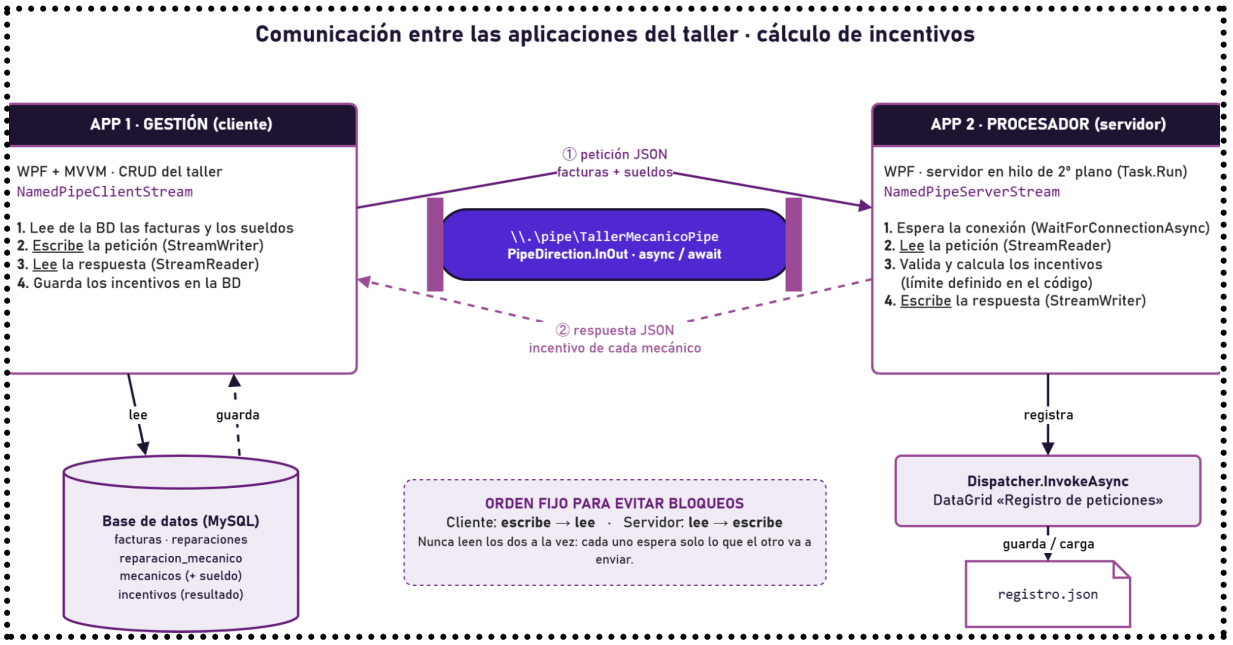


Figura 1. *Esquema de comunicación entre la App 1 y la App 2.*

4.1. Funcionamiento paso a paso

Paso	App 1 · Gestión (cliente)	App 2 · Procesador (servidor)
1	Cuando el usuario de la **App 1** quiere calcular los incentivos, la **App 1** lee de la BBDD las facturas del período y el sueldo fijo de cada empresario. Después se conecta por tubería y envía estos datos a JSON.	Se abre primero y deja la tubería esperando conexiones en un hilo de segundo plano, para que la ventana no se congele.
2	Guarda este resultado en la BBDD.	Recibe la petición y suma lo facturado por cada mecánico. Lo compara con el límite que tiene en su código (como cuando se supera los incentivos en una tienda de ropa, que se prefija en un número a partir de las expectativas de lo que se va a ganar ese mes); después calcula el incentivo y devuelve el resultado en JSON
3	La comunicación es **InOut** (la misma tubería sirve para ir y volver) y **asíncrona** (`async/await`), así que ninguna de las dos apps se queda bloqueada mientras espera.	

5. FORMATO DE LOS DATOS (JSON)

5.1. JSON (App 1 → App 2)

JSON · PETICIÓN

```
{
  "mecanico": {
    "id_mecanico": 1,
    "nombre": "Carlos García",
    "especialidad": "Mecánica general"
  },
  "reparacion": {
    "id_reparacion": 15,
    "id_vehiculo": 3,
    "fecha": "2026-10-20",
    "descripcion": "Cambio de aceite y filtros",
    "estado": "Finalizada"
  },
  "trabajo": {
    "horas_trabajadas": 2.5,
    "coste_hora": 30.00
  },
  "factura": {
    "id_factura": 25,
    "importe": 75.00,
    "fecha": "2026-10-20",
    "resultado": "Pagada"
  }
}
```

5.2. Respuesta (App 2 → App 1)

JSON · RESPUESTA

```
{
  "id_mecanico": 1,
  "nombre_mecanico": "Carlos García",
  "id_reparacion": 15,
  "incentivo": {
    "concedido": true,
    "importe": 50.00,
    "motivo": "Reparación finalizada correctamente"
  },
  "resultado": "OK"
}
```

5.3. Registro de peticiones

JSON · REGISTRO DE PETICIONES

```
{
    "id_peticion": 101,
    "fecha": "2026-10-26T09:30:00",
    "id_mecanico": 1,
    "nombre_mecanico": "Carlos García",
    "accion": "calcular_incentivo",
    "resultado": "OK",
    "incentivo": 50.00,
    "error": null
  }
```

7. CONCLUSIONES

Las **NamedPipes** en C# sirven para crear hilos entre diferentes apps para comunicarse a través de un JSON y facilitar los procesos; con diferentes atributos como **async y await** se puede evitar el bloquear el hilo mientras se opera con operaciones asíncronas.

El trabajo nos ha dado una idea bastante buena de cómo podríamos desarrollar con mayor profundidad el tema de la conexión entre las 2 apps dentro de nuestro taller mecánico; para el ejemplo que hemos usado de **cálculo de incentivos**, que podría utilizar la gente de *RRHH o de contabilidad* para facilitar muchísimo el tema de los cálculos mensuales de cada trabajador, a parte del suelo con todas las deducciones y percepciones que suelen aparecer.

Otras herramientas existentes como archivos compartidos en **Google Drive o Sheets u otras herramientas** como Notion; son mucho más fáciles de entender pero no avisa cuando hay datos nuevos y si dos apps se usan a la vez da problemas.

8. BIBLIOGRAFÍA

- Microsoft Learn. Espacio de nombres System.IO.Pipes. https://learn.microsoft.com/es-es/dotnet/api/system.io.pipes
- Microsoft Learn. Cómo usar canalizaciones con nombre para la comunicación entre procesos en red. https://learn.microsoft.com/es-es/dotnet/standard/io/how-to-use-named-pipes-for-network-interprocess-communication
- Microsoft Learn. Comunicaciones entre procesos (Win32). https://learn.microsoft.com/es-es/windows/win32/ipc/interprocess-communications
- Microsoft Learn. Programación asincrónica con async y await. https://learn.microsoft.com/es-es/dotnet/csharp/asynchronous-programming/
- Microsoft Learn. Método Dispatcher.InvokeAsync. https://learn.microsoft.com/es-es/dotnet/api/system.windows.threading.dispatcher.invokeasync
- Microsoft Learn. Serialización y deserialización de JSON en .NET (System.Text.Json). https://learn.microsoft.com/es-es/dotnet/standard/serialization/system-text-json/overview
- Stack Overflow. Ejemplo de cliente con NamedPipeClientStream (CC BY-SA 4.0). https://stackoverflow.com/q/50597091
- Enunciado del Desafío PSP y DIN, 1.er trimestre, curso 2026-2027.